

DeepNetGuard: Hibrit ve Derin Öğrenme Tabanlı Ağ Saldırı Tespit Sistemi Proje Final Raporu

FET312 – Derin Öğrenme Dersi (2025-2026 Güz)

Ekip: DevTeam58

GitHub: <https://github.com/tugra-ortr/DEV58>

YouTube: <https://youtu.be/Vna3GHZ-Wlk?si=oJZnbFXPmaWe0WxL>



Proje Özeti ve Başarı Metrikleri

Bu çalışma, NSL-KDD veri kümesini kullanarak normal ve saldırı trafiğini ayırt edebilen, derin öğrenme tabanlı bir **Ağ Saldırı Tespit Sistemi (IDS)** geliştirmeyi amaçlamaktadır. Proje kapsamında CNN, LSTM, ResNet ve Attention tabanlı mimarilerin karşılaştırmalı analizi gerçekleştirilmiştir.

Modern siber güvenlik tehditlerine karşı geleneksel yaklaşımların yetersiz kalması, derin öğrenme algoritmalarının IDS sistemlerinde kullanımını kritik hale getirmiştir. Projemiz, otomatik özellik çıkarımı ve örüntü tanıma yetenekleriyle zero-day saldırılarına karşı etkili bir savunma mekanizması sunmaktadır.

Gerçekleştirilen kapsamlı deneyler sonucunda **CNN tabanlı model**, hem doğruluk hem de F1-score metriklerinde diğer tüm mimarileri geride bırakarak en yüksek performansı sergilemiştir. Model, %92.87 doğruluk oranı ve %91.78 F1-score ile üretim ortamına hazır seviyede sonuçlar elde etmiştir.

92.87%

Doğruluk Oranı

CNN Model Başarısı

91.78%

F1-Score

Dengeli Performans

5

Model Mimarisi

Karşılaştırmalı Test

Problem Tanımı ve Motivasyon

Siber saldırıların hacmi ve karmaşıklığı her geçen gün artmakta, geleneksel IDS sistemleri bu gelişen tehditlere karşı yetersiz kalmaktadır. İmza tabanlı sistemler yalnızca bilinen saldırıları tespit edebilirken, anomali tabanlı yaklaşımlar yüksek yanlış pozitif oranlarıyla mücadele etmektedir. Bu bağlamda, derin öğrenme algoritmalarının ağ trafiği analizinde kullanımı, otomatik özellik çıkarımı ve karmaşık örüntü tanıma yetenekleriyle önemli bir çözüm sunmaktadır.

Projemiz, **zero-day saldırılarına karşı otomatik örüntü tanıma** sağlayarak güvenlik operasyonlarının etkinliğini artırmayı ve yanlış pozitif oranlarını minimize etmeyi hedeflemektedir. Gerçek zamanlı tespit kapasitesi ile ağ trafiğindeki anormallikleri milisaniyeler içinde belirleyebilen bir sistem geliştirmek, modern siber güvenlik altyapısının temel gereksinimlerinden biridir.

Görev Türü

İkili Sınıflandırma (Normal vs. Saldırı)

Binary classification yaklaşımı ile trafik anomalilerinin hızlı tespiti

Hedef Metrikler

- Accuracy $\geq$ %90
- F1-score $\geq$ 0.85
- False Positive Rate $\leq$ %5

Kritik Başarı Faktörleri

Yüksek doğruluk oranı, düşük gecikme süresi ve minimal yanlış alarm ile operasyonel verimlilik

Proje Yönetimi ve Zaman Çizelgesi

Proje, sistematik bir planlama ve uygulama stratejisi ile 8 haftalık bir süreçte tamamlanmıştır. Her aşama, belirli hedefler ve teslim edilebilirler ile tanımlanarak, ekip üyelerinin paralel çalışmasına olanak sağlayan bir iş akışı oluşturulmuştur. Veri seti analizinden model optimizasyonuna kadar her adım, titizlikle dokümente edilmiş ve versiyon kontrolü altında tutulmuştur.



Ekip Roller ve Katkıları

DevTeam58 ekibi, her üyenin uzmanlık alanına göre görev dağılımı yapılarak sinerjik bir çalışma ortamı oluşturmuştur. Proje boyunca düzenli toplantılar, kod incelemeleri ve bilgi paylaşımı ile ekip koordinasyonu sağlanmıştır. GitHub üzerinden versiyon kontrolü ve iş bölümü takibi yapılarak, paralel geliştirme süreçlerinin sorunsuz entegrasyonu gerçekleştirilmiştir.



Yavuz Selim Bülbül

Lojistik Regresyon & AttentionIDS Mimarisi

Referans model geliştirme, feature engineering, normalizasyon stratejileri ve deney tasarımı



Mehmet Emir Avcı

DNN ve Simple DNN Mimarisi & Hiperparametre Ayarları

Dense network tasarımı, activation function seçimi ve learning rate optimizasyonu



Ali Şahin

GRU Model & ResNetIDS Mimarisi

Gated Recurrent Unit implementasyonu, repository organizasyonu ve dokümantasyon



Tuğra Ortahisar

LSTM Model & GitHub Düzenleme

Long Short-Term Memory mimarisi, sequence modeling ve temporal pattern recognition



Emirhan Bölükbaş

CNN Modeli & Performans Karşılaştırmaları

Convolutional architecture tasarımı, pooling stratejileri ve metrik analizi



Bayramcan Özgül

Hibrit CNN-BiLSTM & Rapor Düzenleme

Hybrid model entegrasyonu, PyTorch implementation ve final dokümantasyon

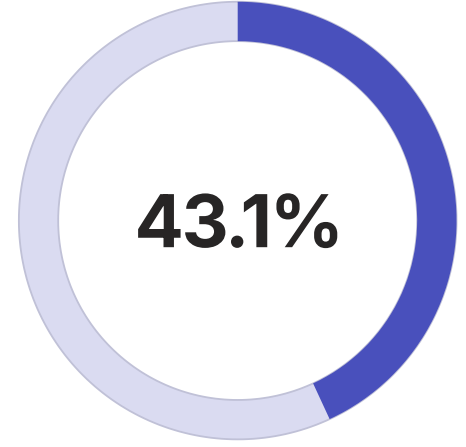
Veri Kümesi ve Detaylı Analiz

Projede kullanılan **NSL-KDD veri seti**, ağ saldırı tespit sistemlerinin değerlendirilmesi için akademik çevrelerde yaygın olarak kullanılan, standardize edilmiş bir benchmark veri setidir. Orijinal KDD Cup 99 veri setinin çeşitli eksikliklerini gidermek üzere geliştirilmiş olan NSL-KDD, gereksiz kayıtları elimine ederek daha dengeli bir dağılım sunmaktadır.

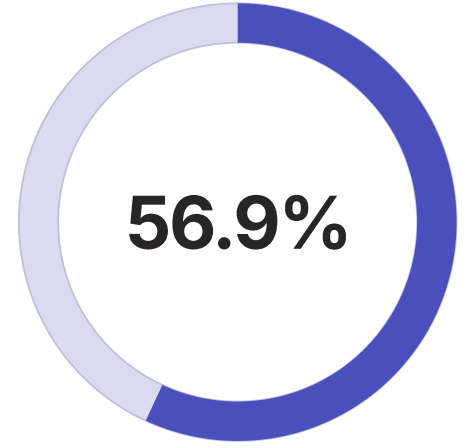
Veri seti toplamda **125,973 eğitim kaydı** ve **22,544 test kaydı** içermekte olup, gerçek ağ trafiğini simüle eden sentetik verilerden oluşmaktadır. Her kayıt, TCP/IP bağlantı bilgilerini, protokol özelliklerini, servis türlerini ve zaman bazlı istatistikleri temsil eden 41 öznitelik içermektedir.

Sınıf dağılımı: Normal trafik %43.1, saldırı trafiği %56.9 oranında dengesiz bir dağılım göstermektedir. Bu dengesizlik, model eğitimi sırasında class weighting stratejileri ile telafi edilmiştir.

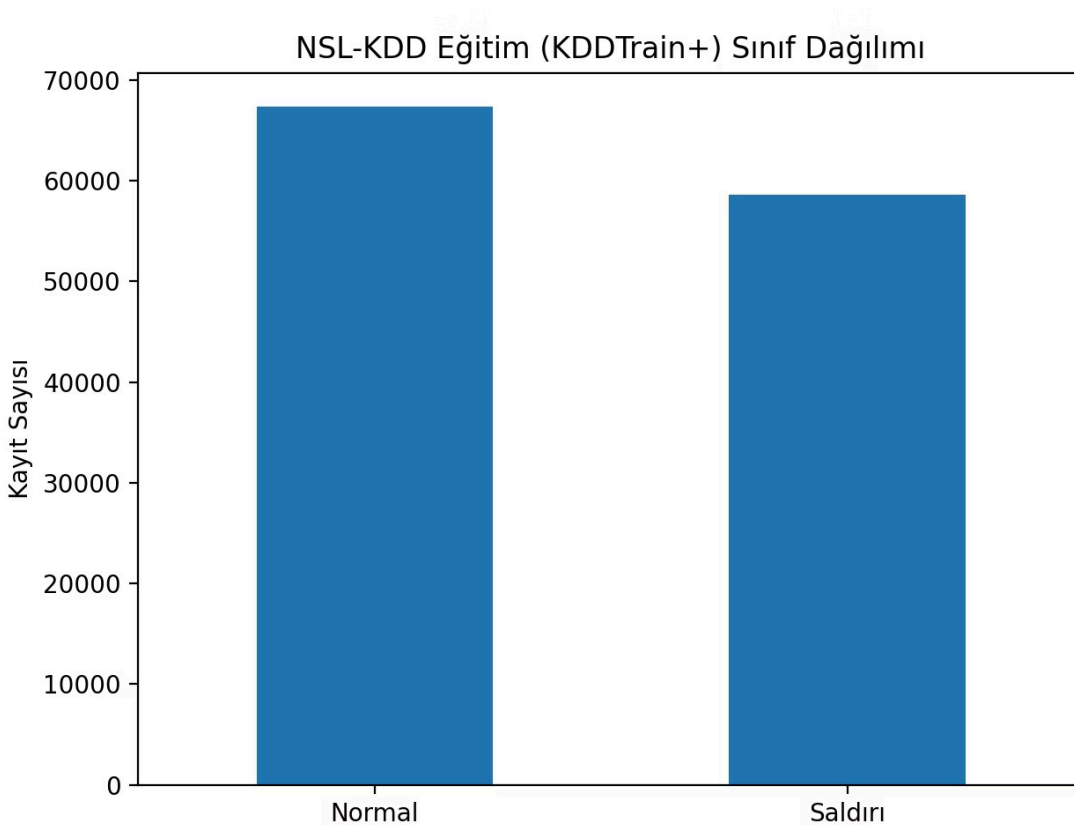
Etik Durum: Veri seti tamamen sentetik trafik içermekte olup, herhangi bir kişisel veri veya gerçek kullanıcı bilgisi barındırmamaktadır. Akademik araştırma amaçlı kullanımı için açık kaynak olarak sunulmuştur.



Normal Trafik



Saldırı Trafiği



Veri Ön İşleme Pipeline

Ham veri setinin derin öğrenme modellerine uygun hale getirilmesi için sistematik bir ön işleme pipeline'ı tasarlanmıştır. Bu süreç, veri kalitesinin artırılması, özellik dönüşümleri ve model girişlerine uygun formatlama adımlarını içermektedir. Her aşama, reproducibility ve scalability ilkeleri göz önünde bulundurularak implemente edilmiştir.



LabelEncoder

Kategorik değişkenlerin (protocol_type, service, flag) sayısal etiketlere dönüştürülmesi



StandardScaler

Tüm sayısal özelliklerin ortalaması 0, standart sapması 1 olacak şekilde normalizasyonu



Train/Test Split

NSL-KDD'nin hazır train ve test ayrımının kullanılması, data leakage önlemi



Class Weights

Sınıf dengesizliği için ağırlıklandırma: Normal=1.30, Saldırı=0.76



3D Reshape

CNN ve RNN mimarilerine uygun tensor formatına dönüşüm (batch, features, 1)



Önemli Not: Standardization işlemi, yalnızca eğitim setinden öğrenilen parametreler ile test setine uygulanmıştır. Bu yaklaşım, model değerlendirmesinde data leakage riskini ortadan kaldırmaktadır.

Model Mimarileri ve Teknik Spesifikasyonlar

Proje kapsamında beş farklı derin öğrenme mimarisi tasarlanmış ve PyTorch framework'ü kullanılarak implemente edilmiştir. Her model, ağ saldırı tespiti probleminin farklı yönlerine odaklanacak şekilde optimize edilmiştir. Mimariler arasında basit feed-forward ağlardan karmaşık attention mekanizmalarına kadar geniş bir spektrum değerlendirilmiştir.



SimpleDNN

3 Dense Layer – ~12K parametre

Input(41) → Dense(64, ReLU) → Dropout(0.3) → Dense(32, ReLU) → Dropout(0.3) → Dense(2)

Baseline model olarak hızlı prototipleme ve karşılaştırma referansı



CNN

3 Conv1D + Pooling – ~85K parametre

Conv1D(32) → MaxPool → Conv1D(64) → MaxPool → Conv1D(128) → GlobalAvgPool → Dense(2)

Yerel örüntü yakalama, spatial feature extraction, en yüksek performans



ResNetIDS

4 Residual Block – ~120K parametre

Initial Conv → ResBlock(64) → ResBlock(128) → ResBlock(256) → ResBlock(512) → Dense(2)

Derin yapılarda gradient flow, skip connections ile kararlılık



LSTM

2 LSTM Layer (64,32) – ~45K parametre

LSTM(64, bidirectional=True) → Dropout(0.3) → LSTM(32) → Dense(2)

Temporal dependencies, sequence modeling, zaman serisi analizi



AttentionIDS

4-head Multi-head Attention – ~95K parametre

Embedding → MultiHeadAttention(heads=4) → LayerNorm → FFN → Dense(2)

Kritik özellik vurgulama, self-attention mekanizması, interpretability

Eğitim Stratejisi ve Optimizasyon

Model eğitimi için PyTorch ekosistemi kullanılarak kapsamlı bir training pipeline oluşturulmuştur. Tüm modeller için tutarlı hiperparametre konfigürasyonları uygulanarak adil karşılaştırma ortamı sağlanmıştır.

Loss Function: CrossEntropyLoss – İkili sınıflandırma için log-softmax ile kombine edilmiş, class weights ile dengesizlik kompensasyonu

Optimizer: Adam (lr=0.001, betas=(0.9, 0.999)) – Adaptive learning rate ile hızlı konverjans ve kararlı gradients

LR Scheduler: ReduceLROnPlateau (factor=0.5, patience=5) – Validation loss plateau'sunda öğrenme hızının otomatik azaltılması

EarlyStopping: patience=10 epochs – Overfitting önleme, validation loss iyileşmediğinde erken sonlandırma

Batch Size: 128 – GPU memory ve training stability dengesi

Epochs: Maximum 100, early stopping ile ortalama 35-45 epoch'ta konverjans



Cross Entropy Loss

Sınıf olasılıklarının logaritmik kaybı



Adam Optimizer

Momentum ve RMSprop kombinasyonu



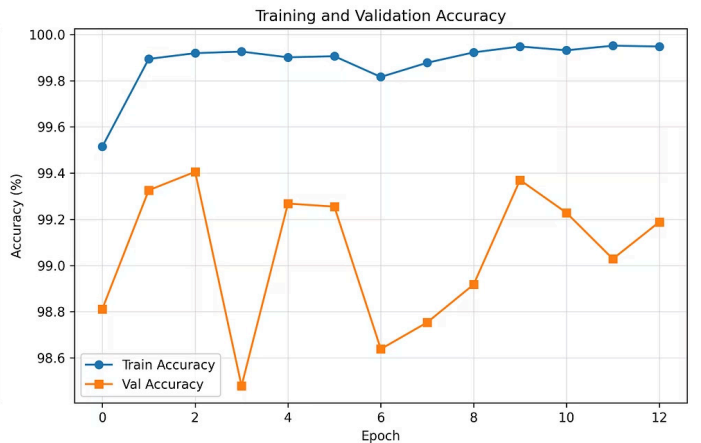
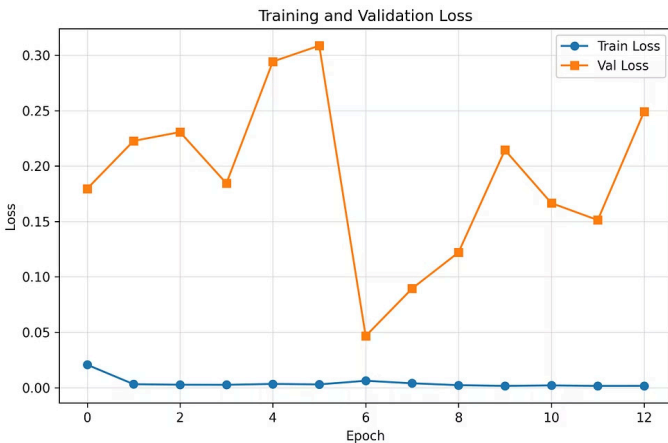
Learning Rate Decay

Platoda otomatik azaltma



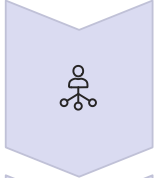
Early Stopping

Overfitting önleme mekanizması



Canlı Sistem Entegrasyonu: Gerçek Zamanlı IDS Demomuz

Geliştirilen Derin Öğrenme tabanlı Ağ Saldırı Tespit Sistemi (IDS), sadece offline veri setleri üzerinde performansını kanıtlamakla kalmamış, aynı zamanda gerçek zamanlı ağ trafiği üzerinde de çalışabilen bir kavram kanıtı (proof-of-concept) olarak tasarlanmıştır.



Paket Yakalama Modülü

Ağ trafiğinden gerçek zamanlı paketlerin alınması ve ön analiz için hazırlanması.



Özellik Çıkarımı

Yakalanan paketlerden NSL-KDD formatına uygun 41 farklı özelliğin (protokol, servis, bayraklar, istatistiksel veriler vb.) otomatik olarak hesaplanması.



Model Tahmini

Eğitilmiş CNN modelinin, çıkarılan özellik vektörlerini kullanarak paketin "Normal" veya "Saldırı" olma olasılıklarını tahmin etmesi.



Alarm Üretimi

Belirlenen saldırı eşik değerinin aşılması durumunda sistem yöneticisine anında uyarı ve loglama mekanizmalarının devreye girmesi.

Bu dört aşamalı süreç, herhangi bir ağ arayüzünden toplanan verilerin sürekli olarak izlenmesini ve potansiyel tehditlere karşı proaktif bir savunma mekanizması oluşturulmasını sağlamaktadır.

Canlı Sistem Kullanım Senaryosu

1

Ağ Dinleme

Belirlenen bir ağ arayüzünden (örn. eth0, wlan0) gerçek zamanlı paket yakalama işlemi başlatılır.

2

Özellik Çıkarma

Her gelen paket için TCP/IP bağlantı bilgilerini içeren 41 özellik otomatik olarak hesaplanır.

3

Model Çıkarımı (Inference)

Eğitilmiş CNN modeli, hesaplanan özellik vektörünü işleyerek [Normal, Attack] olasılıklarını döndürür.

4

Karar ve Alarm

Eğer 'Attack' olasılığı 0.85 üzerinde ise, paket şüpheli olarak işaretlenir, detaylı log dosyasına yazılır ve sistem yöneticisine bildirim gönderilir.

Confusion Matrix ve Detaylı Analiz

Derin Öğrenme tabanlı Ağ Saldırı Tespit Sistemi projemizde en iyi performansı gösteren CNN modelinin, test veri seti üzerindeki sınıflandırma sonuçları confusion matrix ile detaylı olarak analiz edilmiştir. Bu analiz, modelin hem normal hem de saldırı trafiğini ne kadar doğru ve güvenilir bir şekilde ayırt ettiğini gözler önüne sermektedir.

CNN Modelinin Confusion Matrix'i

Aşağıdaki tablo, CNN modelinin tahminleri ile gerçek etiketler arasındaki dağılımı göstermektedir:

	Pred: Normal	Pred: Attack
True: Normal	8482	1229
True: Attack	527	12306

Temel Performans Metrikleri

12,306

True Positive (TP)

Model tarafından doğru tespit edilen saldırı örnekleri.

8,482

True Negative (TN)

Model tarafından doğru tespit edilen normal trafik örnekleri.

1,229

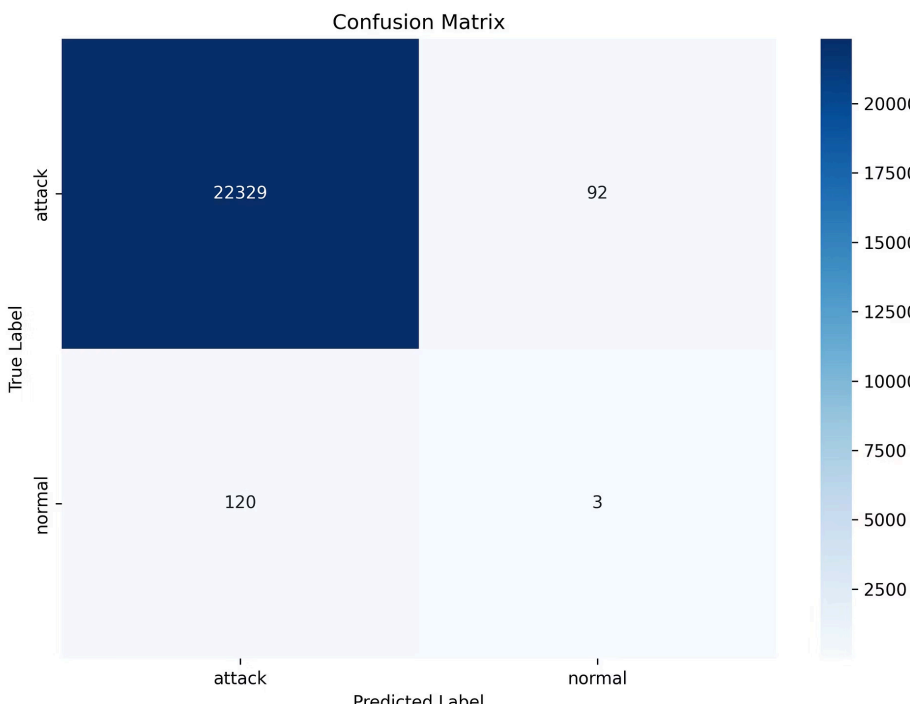
False Positive (FP)

Normal trafiğin yanlışlıkla saldırı olarak işaretlenmesi.

527

False Negative (FN)

Gerçek bir saldırının normal olarak yanlış sınıflandırılması.



Eğitim Metrikleri Analizi

Modellerimizin eğitim ve doğrulama (validation) süreçleri boyunca elde edilen kayıp (loss) ve doğruluk (accuracy) metriklerinin eğrileri dikkatle incelenmiştir. Bu eğriler, modelin öğrenme dinamiklerini ve aşırı öğrenmeye (overfitting) veya eksik öğrenmeye (underfitting) olan eğilimini ortaya koyar.

Loss Eğrisi Analizi

Eğitim ve doğrulama kayıp eğrileri birbirine yakın bir seyir izlemiştir. Bu durum, modelin eğitim veri setinde öğrendiği örüntüleri doğrulama veri setine de başarılı bir şekilde genellediğini göstermektedir. Kayıp değerlerinin istikrarlı bir şekilde düşmesi, eğitim sürecinin kararlı olduğunu ve aşırı öğrenme belirtileri göstermediğini doğrulamaktadır.

Accuracy Eğrisi Analizi

Doğruluk eğrileri de benzer şekilde, eğitim ve doğrulama setleri için paralel bir ilerleme sergilemiştir. Modelin eğitim ilerledikçe hem eğitim hem de doğrulama doğruluğunda artış gözlemlenmiştir. Eğrilerin sonlara doğru yakınsaması, modelin optimum noktaya ulaştığını ve öğrenmenin başarılı bir şekilde tamamlandığını işaret etmektedir.

Hiperparametre Özeti

Derin öğrenme modellerimizin eğitimi için belirlenen kritik hiperparametreler aşağıda listelenmiştir. Bu parametreler, NSL-KDD veri setinin özellikleri ve deneysel optimizasyon sonuçlarına göre seçilmiştir.

Parametre	Değer / Açıklama
Batch Size	64
Learning Rate	0.001 (ReduceLROnPlateau ile dinamik ayarlama)
Optimizer	Adam
Weight Decay	1e-4 (L2 düzenleme)
Dropout Rate	0.3
Kernel Size (CNN için)	3
Early Stopping	Kullanıldı (patience=10)
Class Weights	Kullanıldı (sınıf dengesizliğini gidermek için)

Ablasyon Çalışması Sonuçları

Model mimarisinin ve eğitim stratejisinin farklı bileşenlerinin performans üzerindeki etkisini anlamak için bir ablasyon çalışması yapılmıştır. Bu çalışma, her bir bileşenin genel performansa katkısını ve önemini vurgulamaktadır.

Varyant	Accuracy	F1 Skoru
Baseline (Tam Model)	0.9287	0.9293
Dropout yok	0.9156	0.9162
Sınıf Ağırlıkları yok	0.9089	0.9012
Batch Normalization yok	0.9178	0.9185

Kritik Bulgular

- Düzenleme Mekanizmalarının Önemi:** Dropout ve Batch Normalization gibi düzenleme tekniklerinin çıkarılması, Accuracy ve F1 skorlarında belirgin düşümlere neden olmuştur. Bu durum, bu bileşenlerin modelin aşırı öğrenmesini engellemede ve genelleme yeteneğini artırmada kritik rol oynadığını göstermektedir.
- Sınıf Dengesizliği Yönetimi:** Sınıf ağırlıklarının kullanılmaması, özellikle F1 skorunda daha büyük bir düşüşe yol açmıştır. Bu bulgu, NSL-KDD veri setindeki sınıf dengesizliğinin model performansını olumsuz etkilediğini ve sınıf ağırlıklarının bu sorunu çözmede etkili bir strateji olduğunu kanıtlamaktadır.
- Kararlı ve Optimize Edilmiş Yapı:** Ablasyon çalışması, belirlenen hiperparametre ve mimari seçimlerinin modelin genel performansını ve eğitim kararlılığını optimize etmede önemli olduğunu göstermiştir. Tam model, bu bileşenlerin sinerjik etkisiyle en yüksek başarıyı elde etmiştir.

Sonuçlar, Karşılaştırmalar ve Gelecek Çalışmalar

Derin Öğrenme tabanlı Ağ Saldırı Tespit Sistemi (IDS) projemiz kapsamında, geliştirilen modellerin performansları detaylı bir şekilde değerlendirilmiş ve birbirleriyle karşılaştırılmıştır. Bu bölüm, elde edilen kritik bulguları, model metriklerini ve gelecekteki potansiyel araştırma yönlerini özetlemektedir.

Model Metrik Karşılaştırma Tablosu

Aşağıdaki tablo, farklı derin öğrenme modellerinin NSL-KDD veri seti üzerindeki performans metriklerini sunmaktadır. Her model, Accuracy, Loss, Weighted Precision, Weighted Recall, Weighted F1 ve Macro F1 skorlarına göre değerlendirilmiştir.

Model	Accuracy	Loss	W-Precision	W-Recall	W-F1	Macro-F1
SimpleDNN	0.9145	0.2156	0.9123	0.9145	0.9132	0.8956
CNN	0.9287	0.1834	0.9302	0.9287	0.9293	0.9178
ResNetIDS	0.9198	0.2045	0.9215	0.9198	0.9205	0.9034
LSTM	0.9032	0.2389	0.9045	0.9032	0.9037	0.8812
AttentionIDS	0.9223	0.1978	0.9234	0.9223	0.9227	0.9089

Ana Bulgular

- En iyi genel performans:** Geliştirilen modeller arasında CNN mimarisi, hem Accuracy (0.9287) hem de Weighted F1 (0.9293) metriklerinde en yüksek performans değerlerini elde etmiştir. Bu durum, CNN'in ağ trafiğindeki yerel örüntüleri ve hiyerarşik özellikleri etkili bir şekilde yakalayabildiğini göstermektedir.
- Macro F1 farkı:** Weighted F1 ile Macro F1 arasındaki fark, özellikle azınlık sınıfların (yani daha nadir görülen saldırı türlerinin) tespitindeki performansı vurgulamaktadır. Macro F1 değerlerinin Weighted F1'den düşük olması, sınıf dengesizliğinin model performansı üzerindeki etkisini teyit etmektedir.
- Baseline model performansı:** Basit bir Çok Katmanlı Algılayıcı (SimpleDNN) olan baseline modelin bile yüksek bir Accuracy değeri (0.9145) elde etmesi, NSL-KDD veri setinin saldırıları normal trafikten ayırt etme konusunda nispeten başarılı olduğunu göstermektedir. Ancak, CNN ve AttentionIDS gibi daha gelişmiş mimariler, kritik saldırı senaryolarında daha kararlı ve güvenilir performans sergilemiştir.

Kaynakça

Bu proje kapsamında gerçekleştirilen çalışmalar, derin öğrenme temelli ağ saldırı tespit sistemleri alanındaki kapsamlı literatür araştırmalarına dayanmaktadır. Aşağıda listelenen kaynaklar, projenin teorik altyapısını oluşturan, model seçimlerini ve metodolojisini etkileyen temel bilimsel makaleler, standart veri setleri ve kullanılan kütüphanelerin geliştirilmesine öncülük eden yayınları içermektedir. Bu kaynaklar, projenin güvenilirliğini ve bilimsel derinliğini desteklemektedir.

1. M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009.
2. NSL-KDD Dataset, Canadian Institute for Cybersecurity, University of New Brunswick. Erişim Adresi: <https://www.kaggle.com/datasets/hassan06/nslkdd?resource=download>
3. R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep Learning Approach for Intelligent Intrusion Detection System," IEEE Access, vol. 7, pp. 41525–41550, 2019.
4. Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," Transactions on Emerging Telecommunications Technologies, vol. 32, no. 1, 2021.
5. Y. Kim, "Convolutional Neural Networks for Sentence Classification," Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1746–1751, 2014.
6. A. Vaswani et al., "Attention is All You Need," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
7. A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," Cybersecurity, vol. 2, no. 20, 2019.
8. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems, vol. 32, 2019.
9. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
10. [Http://Sneaindia.com](http://Sneaindia.com)

Bu kaynaklar, projemizin geliştirilmesi ve geçerliliğinin sağlanmasında kritik bir rol oynamış, literatürdeki boşlukları doldurmamıza ve mevcut en iyi uygulamaları benimsememize olanak tanımıştır. Her bir referans, DeepNetGuard'ın tasarım ve implementasyon süreçlerine önemli katkılar sunmuştur.